GUIA VISUAL 062

EXPANSÃO DE PARÂMETROS

*O laço sobre 12 mil arquivos levava **34 minutos**. Ninguém suspeitou do `mv` — o custo eram as outras cinco linhas: `dirname`, `basename`, `echo|sed` e `echo|cut` somam **seis processos por arquivo**. Setenta e dois mil forks, dois milissegundos cada. Trocados por expansão de parâmetros: **seis segundos**.*

062
GUIAS LINUX

ONDE CADA OPERADOR CORTA UM CAMINHO

```
A   B          C   D          E
/ | var/log/apt / | history | .log
```

`${p#*/}`	`var/log/apt/history.log`	menor prefixo até a **1ª** barra — corte **B**
`${p##*/}`	`history.log`	maior prefixo até a **última** barra — corte **D** · é o `basename`
`${arq%.*}`	`history`	tira a extensão — corte **E**
`${p%/*}`	`/var/log/apt`	menor sufixo desde a última barra — corte **C** · é o `dirname`
`${p%%/*}`	`(vazio)`	maior sufixo desde a **1ª** barra — corte **A**
`${arq##*.}`	`log`	só a extensão

Dobrar o símbolo torna o corte guloso, e o padrão é **glob**, não expressão regular: `*`, `?` e `[abc]` existem; `+` e grupos de captura, não. **Os casos-limite divergem dos comandos externos:** sem barra, `${p%/*}` devolve o caminho inteiro (o `dirname` diria `.`); terminando em barra, `${p##*/}` devolve vazio (o `basename` diria `log`); sem ponto, `${p##*.}` devolve tudo. E `${var}` só precisa das chaves para **delimitar o nome**: `$nome_final` é outra variável, `${nome}_final` é o valor mais o sufixo.

VALOR PADRÃO · SUBSTITUIÇÃO · FATIA · CAIXA

OPERADOR	UNSET VAR não definida	VAR="" definida, vazia	VAR="X" com valor
`${var:-p}`	p	p	x
`${var-p}`	p	vazio	x
`${var:=p}`	p · atribui	p · atribui	x
`${var:+a}`	vazio	vazio	a
`${var+a}`	vazio	a	a
`${var:?msg}`	aborta	aborta	x

A coluna do meio é a única em que o dois-pontos muda algo: com ele, definida-e-vazia conta como ausente. Na dúvida use a forma com `:` — variável vazia costuma ser tão inútil quanto inexistente. O `:?` é o único que **interrompe**. E o `:+` resolve o "acrescente só se existir" sem `if`: `comando+="${PORTA:+ -p $PORTA}"`.

`${v/a/b}`	troca a **primeira** ocorrência · `${v//a/b}` troca **todas** · `${v//a}` remove
`${v/#a/b} ${v/%a/b}`	ancora no **início** / no **fim**. `${ip//./-}` → `10-0-0-1`; a barra precisa de escape: `${c//\//-}`
`shopt -s extglob`	acrescenta `+(x)`, `*(x)`, `@(x|y)`, `!(x)` ao glob — ligue **no topo**, nunca no meio: o Bash analisa a sintaxe antes de executar
`${#s} ${s:0:5} ${s: -5}`	comprimento e fatia — **espaço antes do menos**, como nos arrays. `${s:0:-4}` corta do fim
`${n,,} ${n^^} ${n^}`	caixa sem chamar `tr` (Bash 4) — o jeito certo de comparar resposta: `case "${resposta,,}" in s|sim)`
`${!nome} ${!PREFIXO@}`	indireção e listagem por prefixo. **Perigosa com entrada externa** — para tabela, `declare -A`
`${v@Q}`	o valor **citado**, com as fronteiras visíveis (Bash 4.4): `processando 'meu arquivo.txt'` — o formato certo para log

Onde a expansão para e o `awk` começa: ela opera sobre **uma** variável com padrões de glob. Texto com campos e registros, regex de verdade (grupos, quantificadores) ou transformação que depende do contexto entre linhas são trabalho de `grep`, `sed` e `awk` — e aí **chame o programa uma vez sobre o conjunto inteiro**, nunca uma vez por item.

`: "${DESTINO:?variavel DESTINO nao definida}"` **no topo do script é a linha mais barata que existe.** Sem ela, um `DESTINO` vazio por erro de digitação transforma `rm -rf -- "${DESTINO}/cache"` em `rm -rf -- /cache`. O `:` inicial é o comando nulo — não faz nada além de **forçar a avaliação** da expansão; a mensagem vai para `stderr` e o shell sai com código 1, encerrando o script não interativo ali mesmo. Ponha uma linha dessas para **cada** variável obrigatória, antes de qualquer comando com efeito colateral, e mantenha o `--` na chamada.

RESUMO RÁPIDO

- ✓ **Expansão de parâmetros roda dentro do processo do**

NO KALI

O caso mais comum de varredura fora do escopo não é